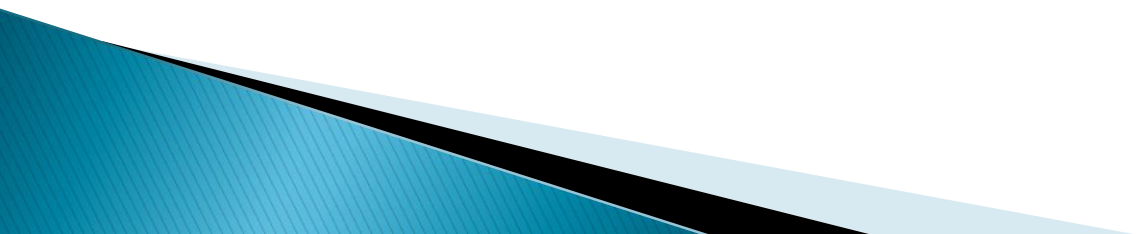
Data Structure

Dr. Tirthankar Gayen
School of Computer & Systems Sciences
Jawaharlal Nehru University



Data Structure

Why is Data Structure important ?

Data Structure

In computer science, a data structure can be considered as a data organization, management, and storage format that enables efficient access and modification.

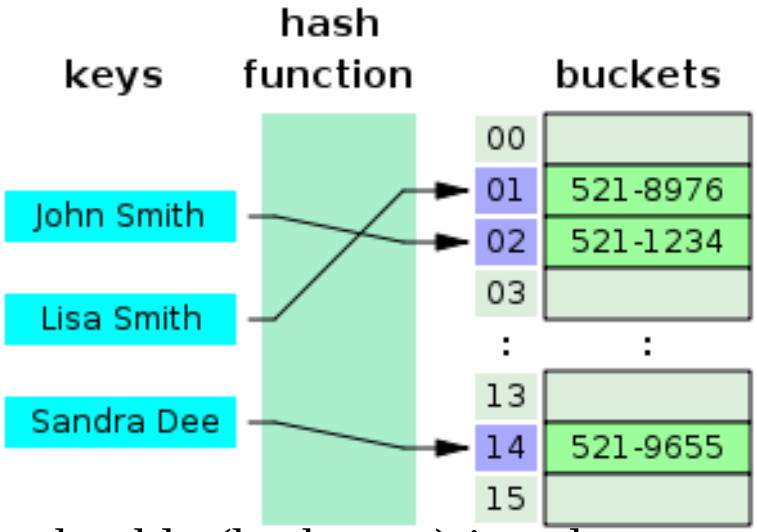
- Cormen, Thomas H.; Leiserson, Charles E.; Rivest, Ronald L.; Stein, Clifford (2009). Introduction to Algorithms, Third Edition (3rd ed.). The MIT Press. ISBN 978-0262033848.
- Black, Paul E. (2004). "data structure". In Pieterse, Vreda; Black, Paul E. (eds.). Dictionary of Algorithms and Data Structures [online]. National Institute of Standards and Technology. Retrieved 2018-11-06.
- "Data structure". Encyclopaedia Britannica. 17 April 2017. Retrieved 2018-11-06.

According to Wegner et al. (2003), a data structure can be considered as collection of data values, the relationships among them, and the functions or operations that can be applied to the data i.e., it can be considered as an algebraic structure about data.

For example: hash table

- Wegner, Peter; Reilly, Edwin D. (2003),Encyclopedia of Computer Science. Chichester, UK: John Wiley and Sons. pp. 507–512.

Data Structure - example



- In computing, a hash table (hash map) is a data structure that implements an associative array abstract data type, a structure that can map keys to values.
- A hash table uses a hash function to compute an index, also called a hash code, into an array of buckets or slots, from which the desired value can be found.
- During lookup, the key is hashed and the resulting hash indicates where the corresponding value is stored.

What is algorithm ?

An algorithm is a finite set of instructions that, if followed, accomplishes a particular task. In addition, all algorithms must satisfy the following criteria.

1. **Input** – Zero or more quantities are externally supplied.
2. **Output** – At least one quantity is produced.
3. **Definiteness** – Each instruction is clear and unambiguous.
4. **Finiteness** – If we trace out the instructions of an algorithm, then for all cases, the algorithm terminates after a finite number of steps.
5. **Effectiveness** – Every instruction must be very basic so that it can be carried out, in principle, by a person using only pencil and paper. It is not enough that each operation be definite as in criterion 3; it also must be feasible.

- A ***program*** is the expression of an algorithm in a programming language.

Pseudocode

- **Pseudocode** is an informal high-level description of the operating principle of a computer program or other algorithm.
- It uses the structural conventions of a programming language, but is intended for human reading rather than machine reading.
- Pseudocode typically omits details that are not essential for human understanding of the algorithm, such as variable declarations, system-specific code and some subroutines.
- The programming language is augmented with natural language description details, where convenient, or with compact mathematical notation.
- The purpose of using pseudocode is that it is easier for people to understand than conventional programming language code, and that it is an efficient and environment-independent description of the key principles of an algorithm.

Recursive Algorithms

- A recursive function is a function that is defined in terms of itself
- Similarly, an algorithm is said to be recursive if the same algorithm is invoked in the body.
- An algorithm that calls itself directly is direct recursive.
- Algorithm A is said to be indirect recursive if it calls another algorithm which in turn calls A.
- Any recursion fundamentally has two parts: the recursive relation and the termination condition(s).

Tail Recursion

- A function call is said to be tail recursive if there is nothing to do after the function returns except return its value.

- Since the current recursive instance is done executing at that point, saving its stack frame is a waste. Specifically, creating a new stack frame on top of the current, finished, frame is a waste.

- A compiler is said to implement TailRecursion if it recognizes this case and replaces the caller in place with the callee, so that instead of nesting the stack deeper, the current stack frame is reused

Discussion

```
Algorithm factorial(n)
{
   if (n == 0) return 1;
   return n * factorial(n - 1);
 }
```

This definition is *not* tail-recursive since the recursive call to factorial is not the last thing in the function (its result has to be multiplied by n)

Discussion

```
Algorithm factorial1(n, accumulator)
{
  if (n == 0) return accumulator;
   return factorial1(n - 1, n * accumulator);
}

Algorithm factorial(n)
{
   return factorial1(n, 1);
}
```

Array

- In computer science, an array data structure, or simply an array, is a data structure consisting of a collection of elements (values or variables), each identified by at least one array index or key.

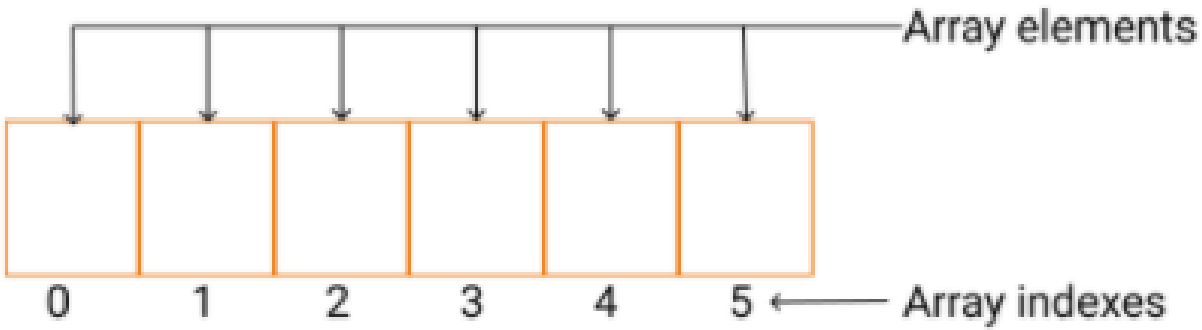
- If one asks a group of programmers to define an array, the most often quoted saying is: a consecutive set of memory locations.

  This is unfortunate because it clearly reveals a common point of confusion, namely the distinction between a data structure and its representation.

- It is true that arrays are almost always implemented by using consecutive memory, but not always.

Ellis Horowitz, Sartaj Sahni, Fundamentals of Data Structures, Computer Science Press, Inc., 1988.

Array CONT.



- **Array Index:** The location of an element in this array has an index, which identifies the element.
- **Array element:** Item stored in an array is called an element. The elements can be accessed via its index.
- **Array Length:** The length of an array is defined based on the number of elements an array can store. In the above example, array length is 6 which means that it can store 6 elements.

Array CONT.

- Normal variables (a1, a2, a3, ….) can be used when we have a small number of elements, but if we want to store a large number of elements, it becomes difficult to manage them with normal variables.

- Representing many elements with one variable name is the basic idea behind arrays.

- When an array of size and type is declared, the compiler allocates enough memory to hold all elements of data.

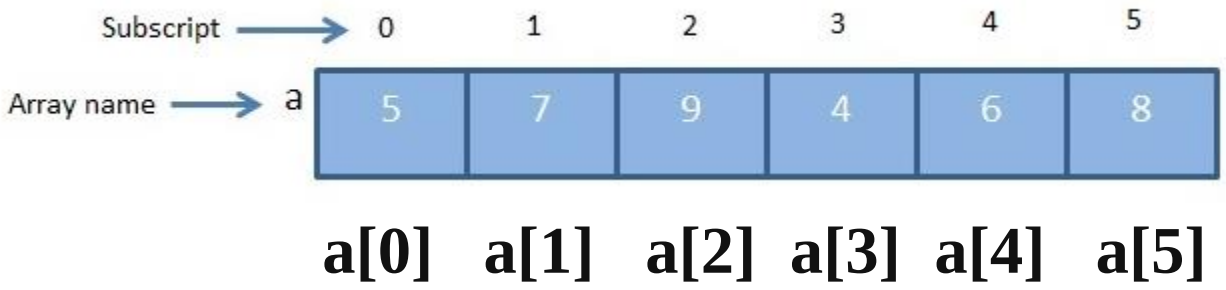
- For example in **C**

  int a[10];

  will have 10 elements with index starting from 0 to 9

Array CONT.

int a[6]= { 5,7,9,4,6,8};

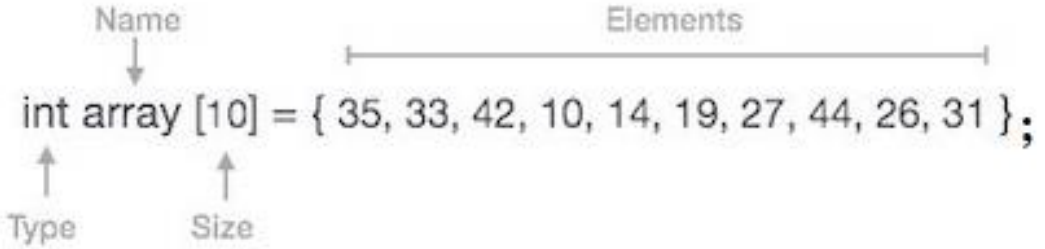


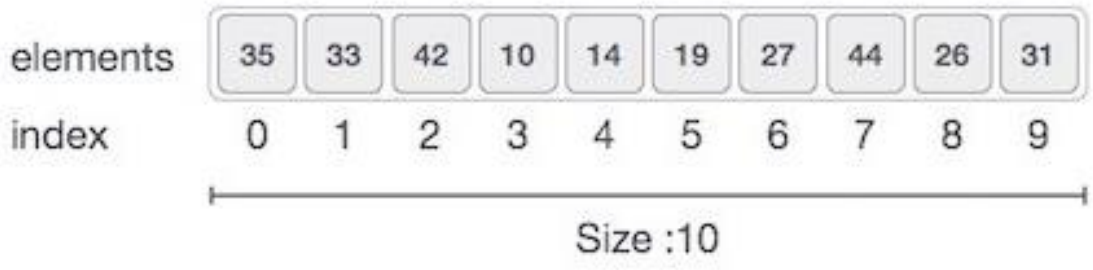
Array CONT.

Following are the important terms to understand the concept of array.

- **Element** − Each item stored in an array is called an element.
- **Index** − Each location of an element in an array has at least an index, which is used to identify the element.

Arrays can be declared in various ways in different languages. For illustration, let's take **C** array (one-dimensional) declaration.





Discussions

- Let ***A(0:n – 1)*** represent an array of size ***n*** beginning at index ***0*** and ending at index ***n – 1***

- Let A(0), A(1), A(2), …, A(n – 1) represent A[0], A[1], A[2], …, A[n – 1] respectively.

One-dimensional arrays

- For a one-dimensional array (or single dimension array), if the valid element indices begin at 0, the constant ***α*** is simply the address of the first element of the array, then with linear addressing scheme, the element with index ***i*** is located at the address ***α*** + ***i*** ×***c***, where ***α*** is a fixed *base address* and ***c*** *is* a fixed constant (may depend on the size of the element) sometimes called the *address increment* or *stride*

A(0),	A(1),	A(2),	…,	A(i),	…,	$A(u_1-1)$
α,	α+c,	α+2c,	…,	α+ic,	…,	$\alpha+(u_1-1)c$

SEQUENTIAL REPRESENTATION OF $A(0:u_1-1)$

- However, if the valid element indices begin at 1, the constant ***α*** is simply the address of the first element of the array, then with linear addressing scheme, the element with index ***i*** is located at the address ***α*** + (***i -1***) × ***c***

A(1),	A(2),	A(3),	…,	A(i),	…,	$A(u_1)$
α,	α+c,	α+2c,	…,	α+(i-1)c,	…,	$\alpha+(u_1-1)c$

SEQUENTIAL REPRESENTATION OF $A(1:u_1)$

INDEXING

Indexes are also called subscripts. When data objects are stored in an array, individual objects are selected by an index. An index *maps* the array value to a stored object.

There are three ways in which the elements of an array can be indexed:

- **0-based indexing (*zero-based indexing*)**
  The first element of the array is indexed by subscript of 0.

- **1-based indexing(*one-based indexing*)**
  The first element of the array is indexed by subscript of 1.

- **n-based indexing (*n-based indexing*)**
  The base index of an array can be freely chosen.

Usually programming languages allowing *n-based indexing* also allow negative index values and other scalar data types like enumerations, or characters may be used as an array index.

INDEXING CONT.

- *Zero-based indexing* is the design choice of many influential programming languages, including C, Java and Lisp
  - This leads to simpler implementation where the subscript refers to an offset from the starting position of an array, so the first element has an offset of zero.

- Some languages (like FORTRAN 77) specify that array indices begin at ***1***, as in mathematical tradition while other languages (like Fortran 90, Pascal and Algol) let the user choose the minimum value for each index.

  - Arrays can have multiple dimensions, thus it is not uncommon to access an array using multiple indices.
  - Thus two indices are used for a two-dimensional array, three for a three-dimensional array, and *n* for an *n*-dimensional array.
  - The number of indices needed to specify an element is called the dimension, dimensionality, or rank of the array.
  - The memory address of the first element of an array is called first address, foundation address, or base address.

REPRESENTATION OF ARRAYS

- Even though multidimensional arrays are provided as a standard data in most high level languages, it is interesting to see how they are represented in memory.
- A memory may be regarded as one dimensional with words numbered from ***1*** to ***m***.

  So, we are concerned with representing ***n*** dimensional arrays in a one dimensional memory.
- While many representations might seem plausible, one must select one in which the location in memory of an arbitrary array element, say $A(i_1,i_2, ...,i_n)$, can be determined efficiently.

  This is necessary since programs using arrays may, in general, use array elements in a random order.
- In addition to being able to retrieve array elements easily, it is also necessary to be able to determine the amount of memory space to be reserved for a particular array.
- Assuming that each array element requires only one word of memory, the number of words needed is the number of elements in the array.
- If an array is declared as $A(l_1:u_1, l_2:u_2, \ldots , l_n:u_n)$, then it is easy to see that the number of elements is

$$\prod_{i=1}^{n} (u_i - l_i + 1)$$

One-dimensional array

- One of the common ways to represent an array is in row major order.
- If A is declared as A(1:u_1), where **α** is the address of A(1), then assuming one word per element, the address of an arbitrary element A(i) is just [redacted]
- It may be represented in sequential memory as

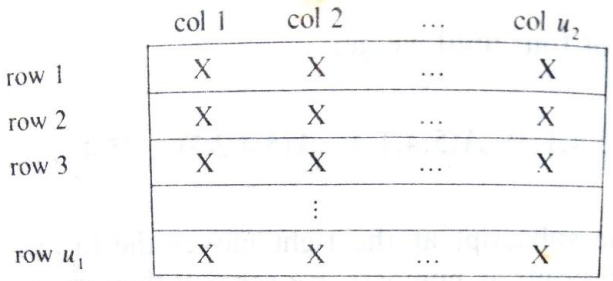
Array element:	A(1),	A(2),	A(3),	…,	A(i),	…,	A(u_1)
Address:	α,	α+1,	α+2,	…,	[redacted],	…,	α+u_1-1

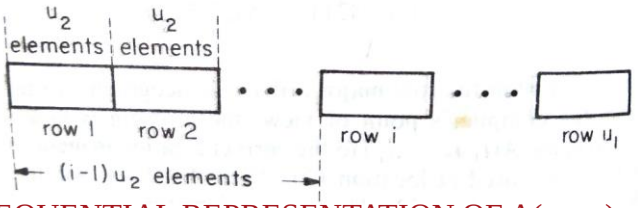
Total number of elements = u_1

SEQUENTIAL REPRESENTATION OF A(1:u_1)

Two-dimensional arrays

- The two dimensional array $A(1:u_1,1:u_2)$ may be interpreted as u_1 rows (row_1, row_2, ...,row_{u1}), where each row consists of u_2 elements.
- In row major representation, these rows can be represented as

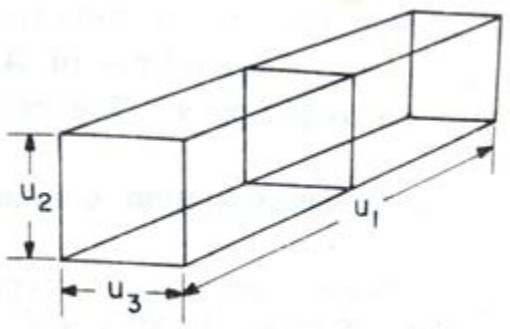




SEQUENTIAL REPRESENTATION OF $A(u_1,u_2)$

- Again, if **α** is the address of A(1,1), then the address of A(i,1) is
- Knowing the address of A(1,1) one can say that the address of A(i,j) is then simply

Three-dimensional array



3-dimensional array regarded as u_1 2-dimensional arrays

- The representation of the 3 dimensional array $A(1{:}u_1,1{:}u_2,1{:}u_3)$ can be interpreted as $\boldsymbol{u_1}$ 2-dimensional arrays of dimension u_2 x u_3.
- To locate A(i,j,k), one can first obtain the address for A(i,1,1) as
- From this and the formula for addressing a 2 dimensional array, one can obtain the address of A(i,j,k) as

n-dimensional array

- Generalizing on the preceding discussion, the addressing formula for any element $A(i_1,i_2, ...,i_n)$ in an n-dimensional array declared as $A(1:u_1,1:u_2,...,1:u_n)$ may be easily obtained.
- If α is the address for $A(1,1, ...,1)$ then the address for $A(i_1,1, ...,1)$ is
- The address for $A(i_1,i_2,1, ...,1)$ is then

n-dimensional array cont.

Repeating in this way the address for $A(i_1,i_2, ...,i_n)$ is

$$
\begin{aligned}
&\alpha + (i_1 - 1)u_2 u_3 \dots u_n \\
&\quad + (i_2 - 1)u_3 u_4 \dots u_n \\
&\quad + (i_3 - 1)u_4 u_5 \dots u_n \\
&\quad \vdots \\
&\quad + (i_{n-1} - 1)u_n \\
&\quad + (i_n - 1) \\
&= \alpha + \sum_{j=1}^{n} (i_j - 1)a_j \quad \text{with} \begin{cases} a_j = \prod_{k=j+1}^{n} u_k & 1 \leq j < n \\ a_n = 1 \end{cases}
\end{aligned}
$$

Discussions

Often the coefficients are chosen so that the elements occupy a contiguous area of memory. However, that is not necessary. Even if arrays are always created with contiguous elements, some array slicing operations may create non-contiguous sub-arrays from them.

There are two systematic compact layouts for a two-dimensional array. For example, consider the matrix

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}.$$

In the row-major order layout (adopted by C for statically declared arrays), the elements in each row are stored in consecutive positions and all of the elements of a row have a lower address than any of the elements of a consecutive row:

1	2	3	4	5	6	7	8	9

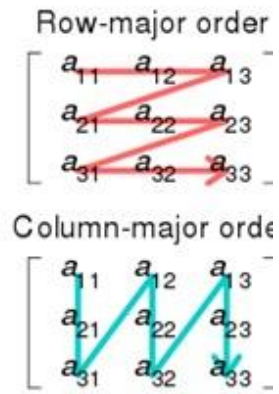


Illustration of row- and column-major order

In column-major order (traditionally used by Fortran), the elements in each column are consecutive in memory and all of the elements of a column have a lower address than any of the elements of a consecutive column:

1	4	7	2	5	8	3	6	9

Assignment

- Obtain an addressing formula for the element $A(i_1, i_2, ..., i_n)$ in an array represented as $A(l_1:u_1, l_2:u_2, ... l_n:u_n)$. Assume a row-major representation of the array (using the sequential byte addressing scheme) with *k* bytes per element, where α is the address of $A(l_1,l_2,...,l_n)$.

- Obtain an addressing formula for the element $A(i_1, i_2, ..., i_n)$ in an array represented as $A(0:u_1, 0:u_2, ... 0:u_n)$. Assume a row-major representation of the array (using the sequential byte addressing scheme) with *k* bytes per element, where α is the address of $A(0,0,...,0)$.

- Obtain an addressing formula for the element $A(i_1, i_2, ..., i_n)$ in an array represented as $A(l_1:u_1, l_2:u_2, ... l_n:u_n)$. Assume a column-major representation of the array (using the sequential byte addressing scheme) with *k* bytes per element, where α is the address of $A(l_1,l_2,...,l_n)$.

How to represent polynomials ?

A general polynomial A(x) can be written as

$$a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x + a_0 \quad \ldots\ldots\ldots\ldots\ldots\ldots \text{(i)}$$

where $a_n \neq 0$ and we say that the degree of A is ***n***.

Then one can represent A(x) as an ordered list of coefficients using a one dimensional array of length ***n + 2***,

$$A = (n, a_n, a_{n-1}, \ldots, a_1, a_0)$$

- The first element is the degree of **A** followed by the ***n + 1*** coefficients in order of decreasing exponent.
- This representation leads to very simple algorithms for addition and multiplication.
- We have avoided the need to explicitly store the exponent of each term and instead we can deduce its value by knowing our position in the list and the degree.

How to represent polynomials ? cont.

But are there any disadvantages to this representation?

- Hopefully you have already guessed the worst one, which is the large amount of wasted storage for certain polynomials.

  Consider $x^{1000} + 1$, for instance.

  - It will require a vector of length 1002, while 999 of those values will be zero.
  -Therefore, we are led to consider an alternative scheme.

- Suppose we take the polynomial A(x) in (i) and keep only its nonzero coefficients. Then we will really have the polynomial

$$b_{m-1}\,x^{e_{m-1}} + b_{m-2}\,x^{e_{m-2}} + \ldots + b_o\,x^{e_o} \qquad \ldots(ii)$$

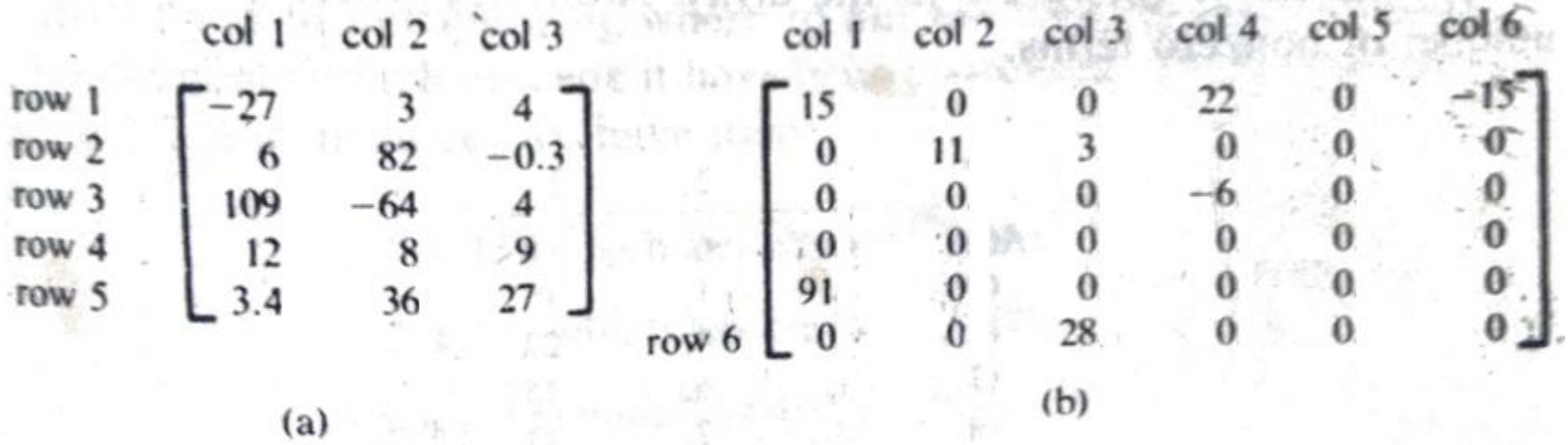
where each b_i is a nonzero coefficient of **A** and the exponents e_i are in the order $e_{m-1} > e_{m-2} > \ldots > e_0 > =0$. If all of **A**'s coefficients are nonzero, then $m = n + 1$, $e_i = i$, and $b_i = a_i$ for $0 <= i <= n$.

Alternatively, only a_n may be nonzero, in which case $m = 1$, $b_0 = a_n$ and $e_0 = n$.

- In general, the polynomial in (ii) could be represented by the ordered list of length $2m + 1$,

$$(m, e_{m-1}, b_{m-1}, e_{m-2}, b_{m-2}, \ldots, e_o, b_o)$$

Martix and Sparse matrix



	col 1	col 2	col 3
row 1	−27	3	4
row 2	6	82	−0.3
row 3	109	−64	4
row 4	12	8	9
row 5	3.4	36	27

(a)

	col 1	col 2	col 3	col 4	col 5	col 6
	15	0	0	22	0	−15
	0	11	3	0	0	0
	0	0	0	−6	0	0
	0	0	0	0	0	0
	91	0	0	0	0	0
row 6	0	0	28	0	0	0

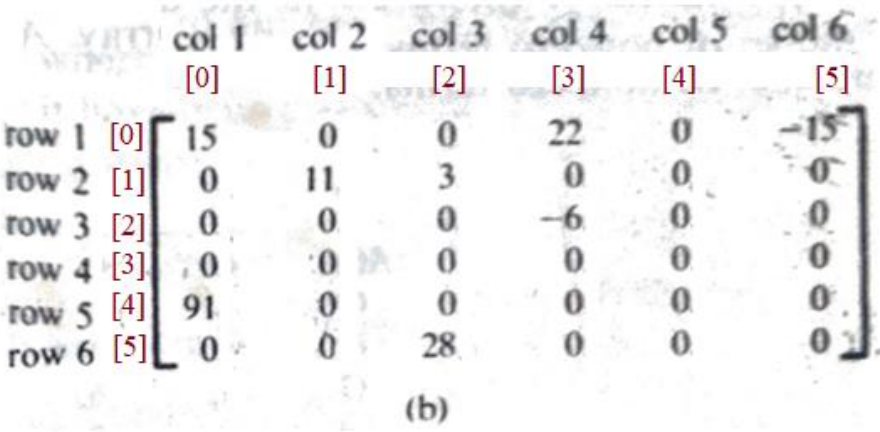
(b)

- A matrix is a mathematical object which arises in many physical problems
- We are interested in studying ways to represent matrices so that the operations to be performed on them can be carried out efficiently.
- The first matrix (shown in fig. a) has five rows and three columns, the second matrix (shown in fig. b) has six rows and six columns.
- In general, we write a matrix of dimenson ***m***x***n*** (read m by n) to designate a matrix with ***m*** rows and ***n*** columns. Such a matrix has **m*n** elements.

  When ***m*** is equal to ***n***, we call the matrix square.

Sparse matrix

- It is very natural to store a matrix in a two dimensional array, say A(0:m-1, 0:n-1), then we can work with any element by writing A(i,j); and this element can be found very quickly.
- Now if we look at the second matrix (specified in fig. b), we see that it has many zero entries.
- In this matrix only 8 out of 36 possible elements are nonzero and that is sparse!
  - Such a matrix is called sparse.



	col 1 [0]	col 2 [1]	col 3 [2]	col 4 [3]	col 5 [4]	col 6 [5]
row 1 [0]	15	0	0	22	0	−15
row 2 [1]	0	11	3	0	0	0
row 3 [2]	0	0	0	−6	0	0
row 4 [3]	0	0	0	0	0	0
row 5 [4]	91	0	0	0	0	0
row 6 [5]	0	0	28	0	0	0

(b)

There is no precise definition of when a matrix is sparse and when it is not, but it is a concept which we can all recognize intuitively.

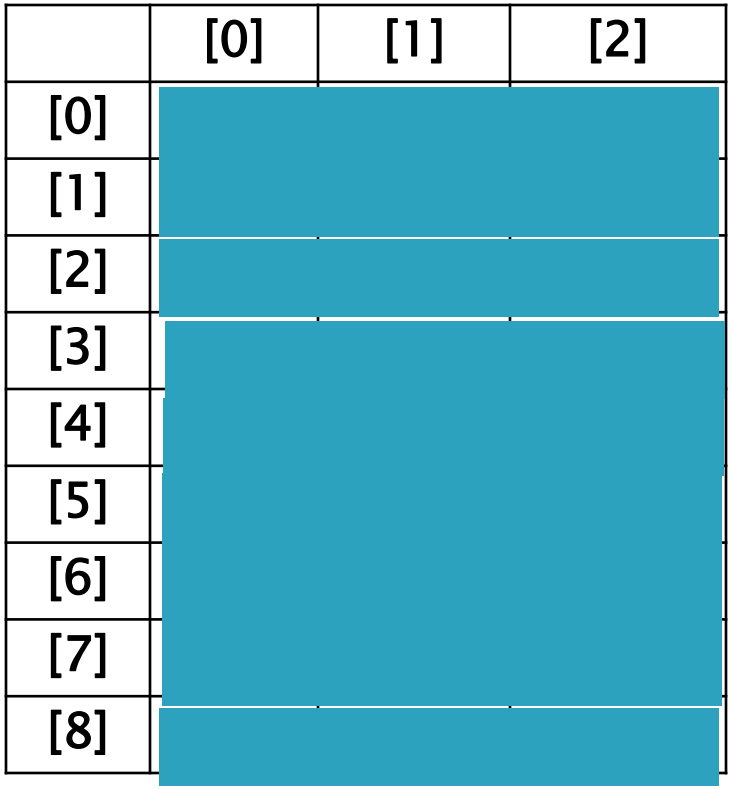
A representation for sparse matrix

- A sparse matrix requires us to consider an alternate form of representation.
- This comes about because in practice many of the matrices we want to deal with are large, (e.g., a matrix of dimension 1000 x 1000), but at the same time they are sparse (say only 1000 out of one million possible elements are nonzero).

Therefore there is a need for an alternative representation for sparse matrices.

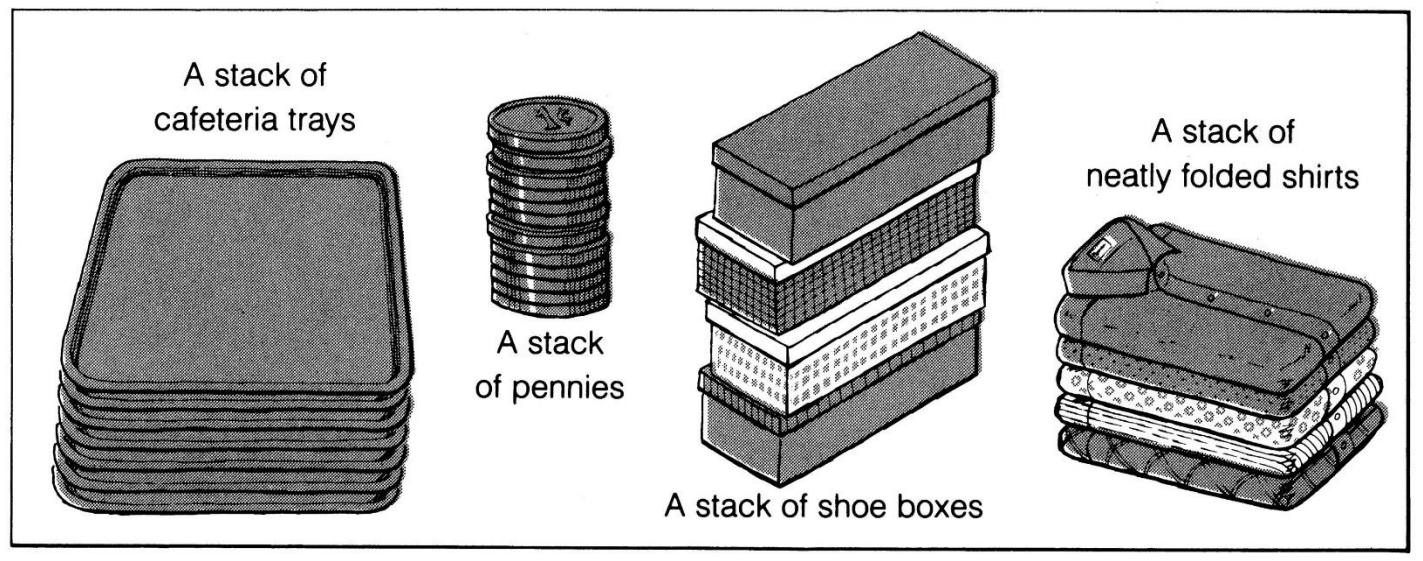
	col 1 [0]	col 2 [1]	col 3 [2]	col 4 [3]	col 5 [4]	col 6 [5]
row 1 [0]	15	0	0	22	0	−15
row 2 [1]	0	11	3	0	0	0
row 3 [2]	0	0	0	−6	0	0
row 4 [3]	0	0	0	0	0	0
row 5 [4]	91	0	0	0	0	0
row 6 [5]	0	0	28	0	0	0

(b)



Stack

- Access is allowed only at one point of the structure, normally termed the *top* of the stack
  - access to the most recently added item only
- Elements are added to and removed from the top of the stack (the most recently added items are at the top of the stack).
- The last element to be added is the first to be removed (**LIFO**: Last In, First Out).



STACK (IMPLEMENTED USING ARRAY) OPERATIONS

```
Algorithm push(item, stack , n, top)
{
   // Insert item into the stack of maximum size n; top refers to the index of the
   topmost element in the stack
    if(top+1 == n)                          //Assuming index starts from 0

          print ("Overflow : the stack is full");
     else
           {
             top = top +1;
             stack[top]= item;
             }
   return top;
   }
```

STACK (IMPLEMENTED USING ARRAY) OPERATIONS CONT.

```
Algorithm pop(stack, top)
{
    // Removes the top element of stack , unless it is empty
    if(top<0)
        print("Underflow: the stack is empty");
    else
          {
              item =stack[top];
               print("The deleted item is" , item);
               top=top-1;
             }
     return top;
    }
```

The Call Stack

- In computer science, a **call stack** is a stack data structure that stores information about the active subroutines/functions of a computer program.

- This kind of stack is also known as an **execution stack**, **program stack**, **control stack**, **run-time stack**, or **machine stack**, and is often shortened to just "**the stack**".

- A **call stack** is composed of **stack frames** (also called *activation records* or *activation frames*).

- These are data structures containing subroutine/function state information.

- Each stack frame corresponds to a call to a subroutine/function which has not yet terminated with a return.

- The stack frame at the top of the stack is for the currently executing routine, which can access information within its frame (such as parameters or local variables).

- The stack frame usually includes at least the following items:
  - the arguments (parameter values) passed to the routine (if any)
  - the return address back to the routine's caller
  - space for the local variables of the routine (if any).

Tower of Hanoi

- The **Tower of Hanoi** (also called **The problem of Benares Temple** or **Tower of Brahma** or **Lucas' Tower** and sometimes pluralized as **Towers**, or simply **pyramid puzzle**) is a mathematical game or puzzle consisting of three rods and a number of disks of various diameters, which can slide onto any rod.
- The puzzle begins with the disks stacked on one rod in order of decreasing size, the smallest at the top.
- The objective of the puzzle is to move the entire stack to the last rod, obeying the following rules:
  - Only one disk may be moved at a time.
  - Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack or on an empty rod.
  - No disk may be placed on top of a disk that is smaller than it.
- The minimal number of moves required to solve a Tower of Hanoi puzzle is $2^n - 1$, where n is the number of disks.
  - With 3 disks, the puzzle can be solved in 7 moves.

```
Algorithm tower(n, source, inter, dest)
{
   if(n>0)
     {
       tower(n-1, source, dest, inter);
       print(“Move disk”, n, “from”, source, “to”, dest);
       tower(n-1, inter, source, dest);
     }
 }
```

Queue

Introduction to Queues

- A queue is a **waiting line**

- It's in daily life:
  - A line of persons waiting to check out at a supermarket
  - A line of persons waiting to purchase a ticket for a film
  - A line of planes waiting to take off at an airport
  - A line of vehicles at a toll booth

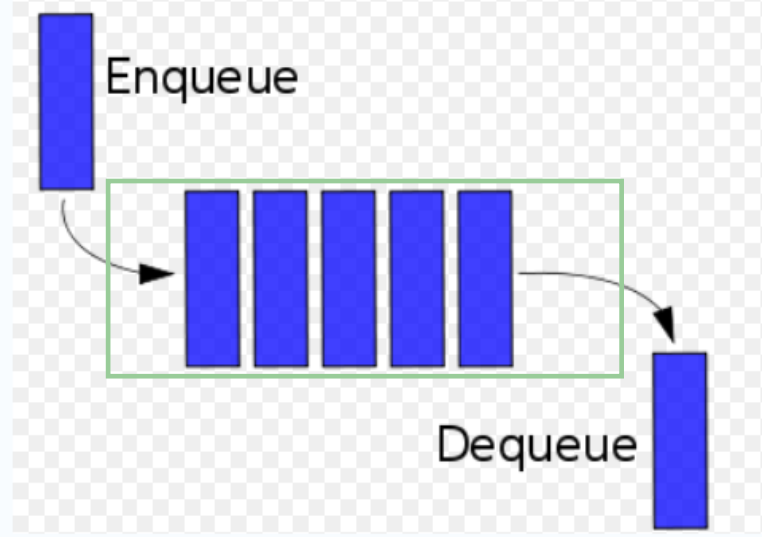
Queues

- One end is called front
- Other end is called rear

- Additions (insertions or enqueue) are done at the rear
- Removals (deletions or dequeue) are made from the front

The Queue

- **Dequeue:**
  - **Removes the item at the front of a non-empty queue.**
- **Enqueue:**
  - **Inserts the item at the rear of the queue**



Operations on queue (an implementation using array)

```
Algorithm enqueue(item, q, n, rear)
{
        // insert item into the queue implemented using array q(0:n-1)
   if (rear = = n-1)
     print("Overflow: the queue is full - insertion is not possible");
   else
    {
     rear = rear+1;
     q[rear] = item;
      }
   return rear;
   }
```

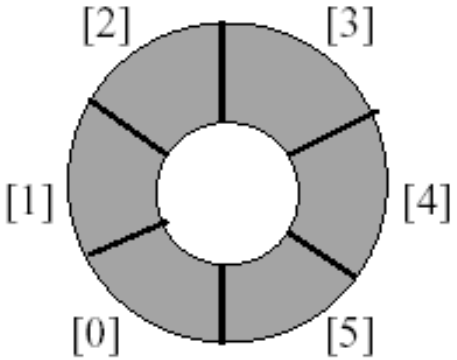
Operations on queue (an implementation using array) cont.

```
Algorithm dequeue(q, front, rear)
{
        // delete an element from queue implemented using array q(0:n-1)
   if (front = = rear)
     print("The queue is empty - deletion is not possible");
   else
    {
      front = front +1;
      item= q[front];
       print("The deleted item is", item);
        }
    return front;
   }
```

Custom Array Queue

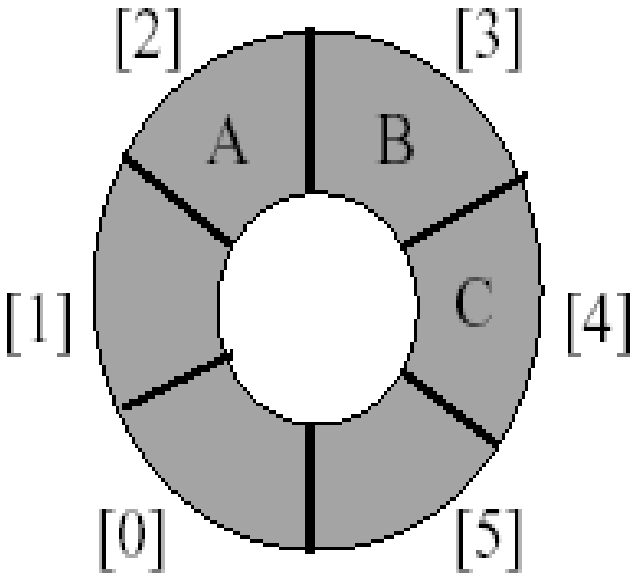
- Use a 1D array queue

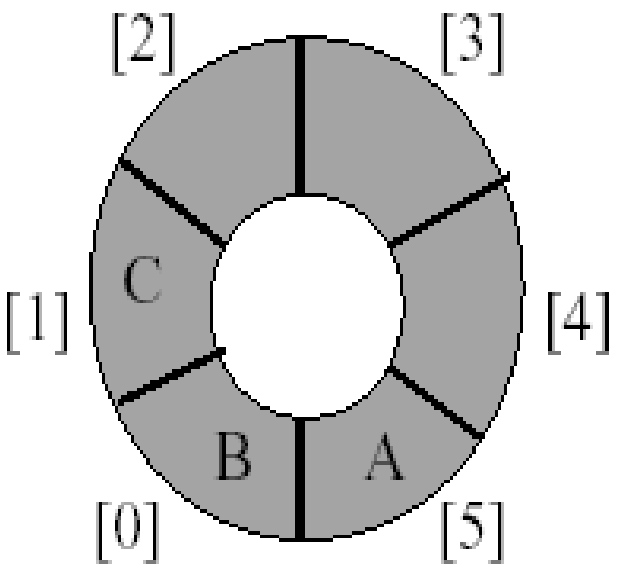
- Circular view of array



Custom Array Queue

Possible configurations with three elements

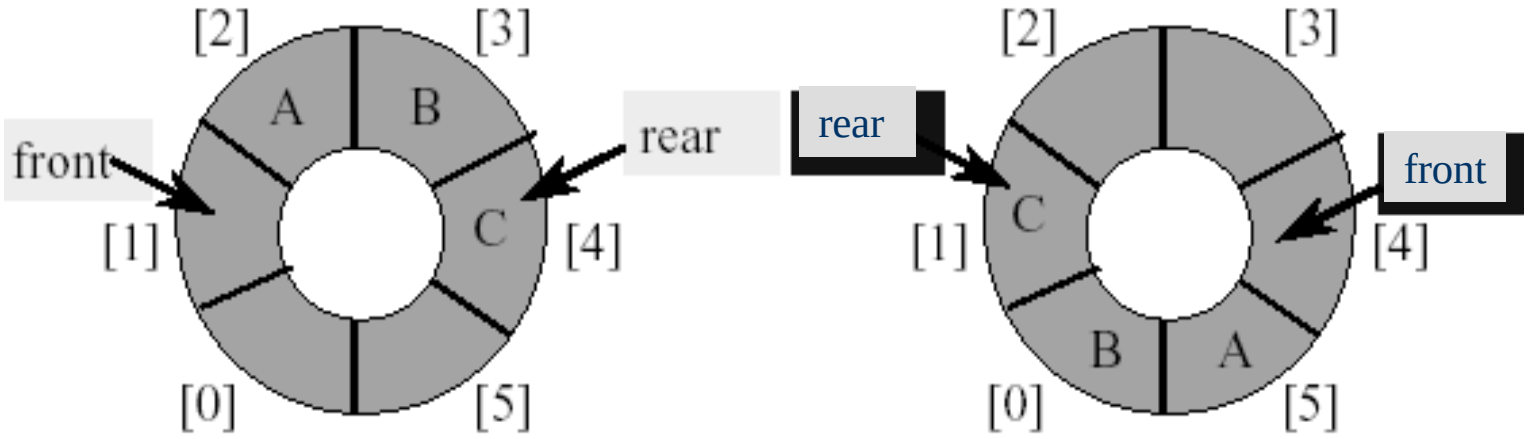




Custom Array Queue

Use integer variables ‘front’ and ‘rear’

- ‘front’ is one position counter-clockwise from first element
- ‘rear’ gives the position of last element



```
Algorithm insertCQ(item, q, n, front, rear)
{
        // insert item into the circular queue implemented using array q(0:n-1)
    temp = (rear+1)% n;
    if (front = = temp )
      print("Overflow: the queue is full - insertion is not possible");
    else
     {
       rear = temp;
       q[rear] = item;
       }
      return rear;
   }
```

```
Algorithm deleteCQ(q, n, front, rear)
{
        // delete an element from circular queue implemented using array q(0:n-1)
       // front and rear are initially initialized to the same value
   if (front = = rear)
     print("The queue is empty - deletion is not possible");
   else
    {
      front = (front +1)% n ;
      item= q[front];
       print("The deleted item is", item);
        }
    return front;
   }
```

Priority Queues

- In a normal queue the enqueue operation add an item at the rear of the queue, and the dequeue operation removes an item at the front of the queue.

- A priority queue is a queue in which the dequeue operation removes an item at the front of the queue; but the enqueue operation inserts item according to their priorities.

- A higher priority item is always enqueued before a lower priority element.

- An element that has the same priority as one or more elements in the queue is enqueued after all the elements with that priority.

Space and Time Complexity

- The *space* complexity of an algorithm is the amount of memory it needs to run to completion.

- The *time* complexity of an algorithm is the amount of computer time it needs to run to completion.

Discussion

The space needed by each of the algorithms is seen to be the sum of the following components:

- A fixed part that is independent of the characteristics (e.g., number size) of the inputs and outputs. This part typically includes the instruction space (i.e., space for the code), space for simple variables and fixed-size component variables (also, called aggregate), space for constants, and so on.
- A variable part that consists of the space needed by component variables whose size is dependent on the particular problem instance being solved, the space needed by referenced variables (to the extent that this depends on instance characteristics), and the recursion stack space.

- The space requirement $S(P)$ of any algorithm P may therefore be written as $S(P) = c + S_p$ (instance characteristics), where c is a constant.
- When analyzing the space complexity of an algorithm, we concentrate solely on estimating S_p (instance characteristics).

Discussion

```
Algorithm abc(a,b,c)
{
  return a+b+b*c+(a+b-c)/(a+b) +4.0;
}
```

The problem instance is characterized by the specific values of **a**,**b** and **c**.

- Assuming that one word is adequate to store the values of each of **a**,**b**,**c** and the result, it is seen that the space needed by algorithm **abc** is independent of the instance characteristics.

Hence, S_{abc}(instance characteristics) = 0

Discussion

```
Algorithm Sum(a,n)
{
  s = 0.0;
  for (i=1; i<= n; i++)
     s=s+a[i];
   return s;
}
```

- The problem instances are characterized by **n**, the number of elements to be summed.
- Assuming that the space needed by **n** is one word
- The space needed by **a** is at least **n** words, since **a** must be large enough to hold the **n** elements to be summed.

Hence $S_{Sum}(n) >=$ 

Discussion cont.

```
Algorithm RSum(a,n)
{
  if(n<=0)  return 0.0;
  else return RSum(a, n-1) + a[n];
}
```

- The recursion stack space usually includes at least space for formal parameters (if any), the local variables (if any) and the return address.
- Assuming that the return address requires only one word of memory.
- Each call to **RSum** requires at least three words (including space for the value of **n**, the return address, and a pointer to **a[]**)
- Since the depth of recursion is **n+1**, the recursion stack space needed is >=

Time Complexity

- The time **T(P)** taken by a program **P** is the sum of the compile time and the run (or execution) time.
- The compile time does not depend on the instance characteristics.
- Also, we may assume that a compiled program will be run several times without recompilation.
- Consequently, we concern ourselves with just the run time of a program.
- This run time is denoted by $\mathbf{t_p}$ (instance characteristics)

Discussion

- In a multiuser multitasking system, the execution time depends on such factors as system load, the number of other programs executing on the computer at the time program **P** is executed, the characteristics of these other programs, and so on.

- Since, many of the factors t_p depends on are not known at the time a program is conceived, it is reasonable to attempt only to estimate t_p in terms of number of program steps count.

- A program step is loosely defined as a syntactically or semantically meaningful segment of a program that has an execution time that is independent of the instance characteristics.

For example, the entire statement

return a+b+b*c+(a+b-c)/(a+b)+4.0; could be regarded as a step since its execution time is independent of the instance characteristics

(This statement is not strictly true, since the time for a multiply and divide generally depends on the numbers involved in the operation.)

Discussion cont.

- The number of steps any program statement is assigned depends on the kind of statement.
- We can determine the number of steps needed by a program to solve a particular problem instance in one of two ways.
- In the first method, we introduce a new variable, count, into the program.
- This is a global variable with initial value 0.
- Statements to increment count by the appropriate amount are introduced into the program.
- This is done so that each time a statement in the original program is executed, count is incremented by the step count of that statement.

Discussion cont.

```
Algorithm Sum(a,n)
{
  s=0.0;
  count=count + 1;                    // count is global: it is initially
zero
  for (i=1; i<=n; i++)
   {
     count = count +1;                // For for
     s= s + a[i]; count= count+1;     // For assignment
    }
     count= count +1;                 // For last time of for
     count= count +1;                 //For the return
     return s;
 }
```

Discussion cont.

```
Algorithm RSum(a,n)
{
  count= count + 1;                          // For the if conditional
  if(n<=0)
    {
      count = count +1;                      // For the return
      return 0.0;
      }
  else
    {
      count = count +1;                      //For the invocation,
addition and return
      return RSum(a, n-1) +a[n];
      }
}
```

$$t_{RSum}(n) = 2 \qquad \text{if } n <= 0$$

$$= 2 + t_{RSum}(n-1) \qquad \text{if } n > 0$$

$$t_{RSum}(n) = 2 + t_{RSum}(n-1)$$

$$= 2 + 2 + t_{RSum}(n-2)$$

$$= 2 + 2 + 2 + t_{RSum}(n-3)$$

$$= 3(2) + t_{RSum}(n-3)$$

.

.

$$= n(2) + t_{RSum}(0)$$

$$= 2n + 2 \qquad \text{when } n >= 0$$

So the step count for RSum is 2n+2

Discussion cont.

- The step count is useful in that it tells us how the run time for a program changes with changes in the instance characteristics.

- For the step count for RSum which is 2n+2, the run time grows linearly in n.

We can say that RSum is a linear time algorithm (the time complexity is linear in the instance characteristics of n)

- One of the instance characteristics that is frequently used in literature is the input size.

- The input size of any instance of a problem is defined to be the number of words (or the *number of elements*) needed to describe that instance.

Example – Matrix addition

```
Algorithm
Add(a,b,c,m,n)
{
  for (i=1; i<= m; i++)
     for (j=1; j<= n; j++)
       c[i,j] = a[i,j] + b[i,j];
  }
```

```
Algorithm Add(a,b,c,m,n)
{
  for (i=1; i<= m; i++)
   {
      count=count+1;                    // For 'for (i=1; i<= m; i++)'

      for (j=1; j<= n; j++)
       {
         count=count+1;                 //For 'for (j=1; j<= n; j++)'
          c[i,j] = a[i,j] + b[i,j];
          count=count+1;                //For the assignment
        }
        count=count+1;
                //For the last time of 'for (j=1; j<= n; j++)'
     }
        count=count+1;
                //For the last time of 'for (i=1; i<= m; i++)'
  }
```

Discussion

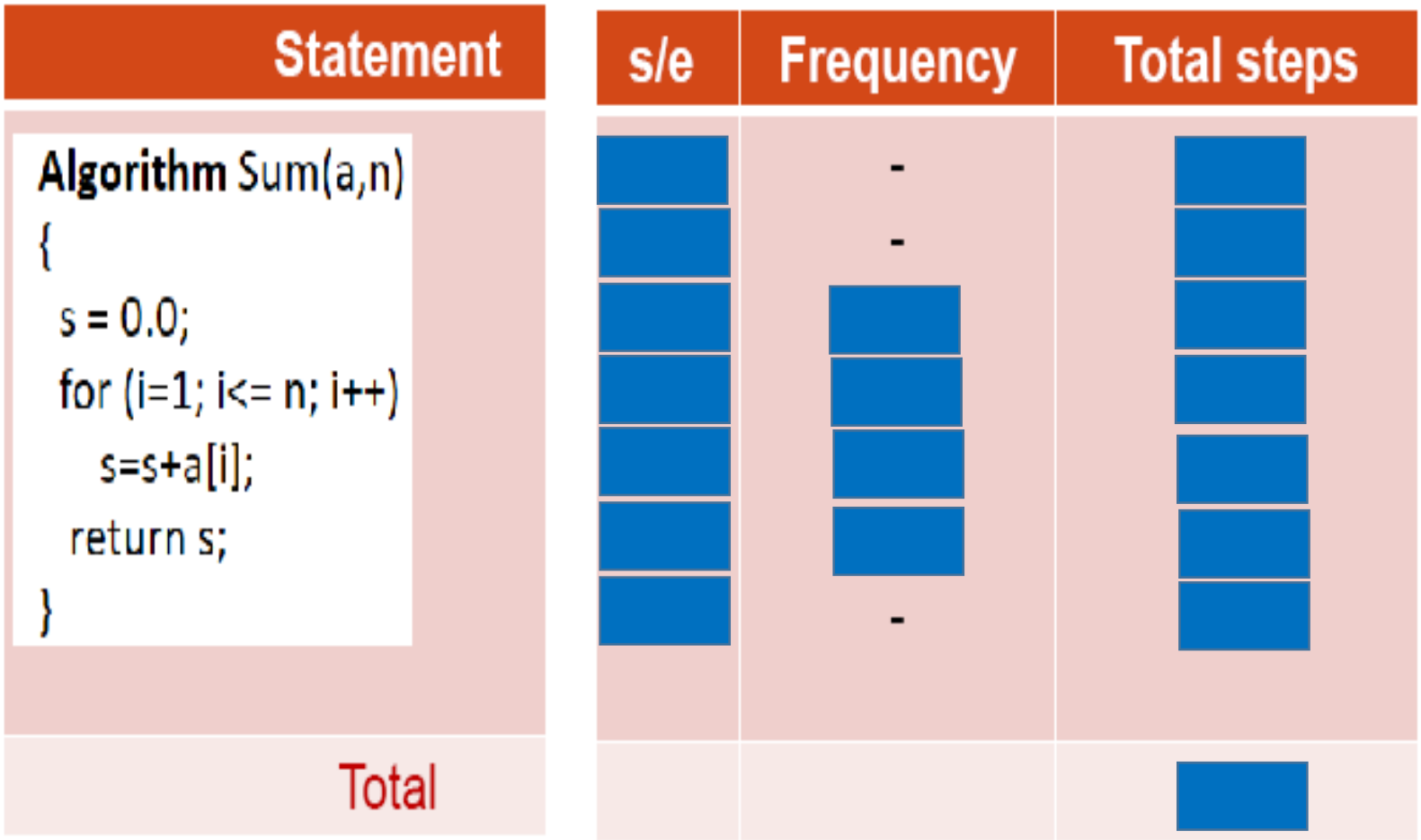
- Another method to determine the step count of algorithm is by first determining the number of steps per execution (s/e) of the statement and then the total number of times (i.e frequency) that statement is executed.

- The s/e of a statement is the amount by which the count changes as a result of the execution of that statement.

- By combining s/e and frequency of execution of each statement , the total contribution of each statement is obtained.

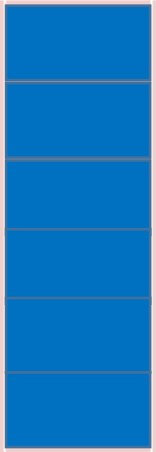
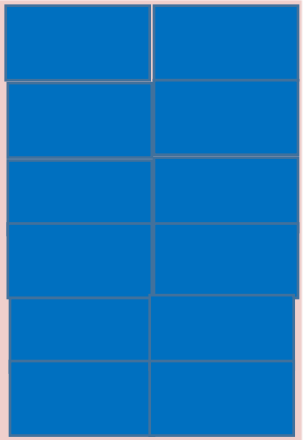
- By adding the contributions of all statements, the step count for the entire algorithm is obtained.

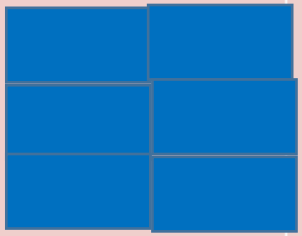
Discussion cont.



Discussion cont.

Statement	s/e	Frequency n=0	Frequency n>0	Total steps n=0	Total steps n>0

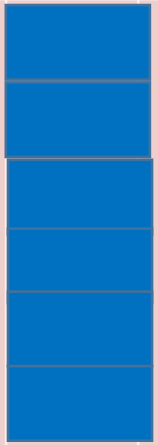
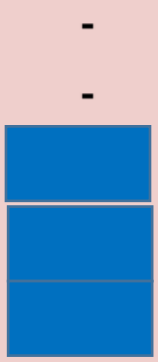
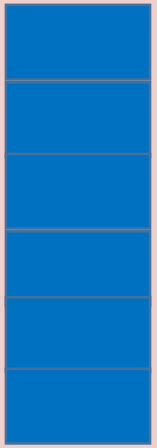
Algorithm RSum(a,n)
{
if(n<=0)
return 0.0;
else return RSum(a, n-1) + a[n];
}
		-	-		

		-	-		
					
		-	-		
Total					

where x= 

Discussion cont.

Statement	s/e	Frequency	Total steps

Algorithm Add(a,b,c,m,n)
{
for (i=1; i<= m; i++)
for (j=1; j<= n; j++)
c[i,j] = a[i,j] + b[i,j];
}
			

Total			

Discussions

- The *best-case* step count is the minimum number of steps that can be executed for the given parameters.
- The *worst-case* step count is the maximum number of steps that can be executed for the given parameters.
- The *average* step count is the average number of steps executed on instances with the given parameters.

Discussions

- If we have two algorithms with a complexity of $c_1n^2+c_2n$ and c_3n respectively, then we know that the one with complexity c_3n will be faster than the one with complexity $c_1n^2 +c_2n$ for sufficiently large values of n.
- No matter what the value of $c1$, $c2$ and $c3$, there will be an n beyond which the algorithm with complexity c_3n will be faster than the one with complexity $c_1n^2 +c_2n$.
- This value of n is called the break-even point.
- If the break-even point is zero, then the algorithm with complexity c_3n is always faster (or at least as fast).
- The exact break-even point cannot be determined analytically.
- The algorithms have to be run on a computer in order to determine the break-even point.

Asymptotic analysis

In mathematical analysis, **asymptotic analysis**, also known as **asymptotics**, is a method of describing limiting behavior.

As an illustration, suppose that we are interested in the properties of a function $f(n)$ as n becomes very large.

- If $f(n) = n^2 + 3n$, then as n becomes very large, the term $3n$ becomes insignificant compared to n^2.
- The function $f(n)$ is said to be "*asymptotically equivalent* to n^2, as $n \to \infty$".

Asymptotic Notation

- ***Big O*** notation is a mathematical notation that describes the limiting behavior of a function when the argument tends towards a particular value or infinity.
- ***Big O*** is a member of a family of notations invented by Paul Bachmann, Edmund Landau, and others, collectively called **Bachmann–Landau notation** or **asymptotic notation**.
- The letter ***O*** was chosen by Bachmann to stand for *Ordnung*, meaning the order of approximation.
- In computer science, ***big O*** notation is used to classify algorithms according to how their run time or space requirements grow as the input size grows.
- ***Big O*** notation is also used in many other fields to provide similar estimates.
- The letter *O* is used because the growth rate of a function is also referred to as the **order of the function**.
- A description of a function in terms of ***big O*** notation usually only provides an upper bound on the growth rate of the function.

Asymptotic Notation

Let f(n) and g(n) be functions from the set of nonnegative integers to the set of nonnegative real numbers.

[Big "oh"] The function $f(n) = O(g(n))$ (read as "f of n is big oh of g of n") iff (if and only if) there exist positive constants c and n_0 such that

$f(n) <= c * g(n)$ for all n, $n >= n_0$

- The function $3n+2 = O(n)$ as $3n+2 <= 4n$ for all $n >= 2$

  $3n+3 = O(n)$ as $3n+3 <= 4n$ for all $n >= 3$

  $100n+6 = O(n)$ as $100n+6 <= 101n$ for all $n >= 6$

$10n^2+4n+2 = O(n^2)$ as $10n^2+4n+2 <= 11n^2$ for all $n >= 5$

$1000n^2+100n-6 = O(n^2)$ as $1000n^2+100n-6 <= 1001n^2$ for all $n >= 100$

$6*2^n + n^2 = O(2^n)$ as $6*2^n + n^2 <= 7*2^n$ for all $n >= 4$

$3n+3 = O(n^2)$ as $3n+3 <= 3n^2$ for $n >= 2$

$3n+2 \neq O(1)$ as $3n+2$ is not less than or equal to c for any constant c and all $n >= n_0$

Similarly, $10n^2+4n+2 \neq O(n)$

Discussions

- We write $O(1)$ to mean a computing time is a constant
  $O(n^2)$ is called quadratic
  $O(n^3)$ is called cubic
  $O(2^n)$ is called exponential

- $f(n) = O(g(n))$ states only that $g(n)$ is the upper bound on the value of $f(n)$ for all n, $n >= n_0$

  Note that $n = O(n)$, $n = O(n^{2.5})$, $n = O(n^3)$, $n = O(2^n)$, and so on.

- For the statement $f(n) = O(g(n))$ to be informative, $g(n)$ should be as small a function of n as one can come up with for which $f(n) = O(g(n))$.

- So, while we often say that $3n+3 = O(n)$, we almost never say that $3n+3 = O(n^2)$, even though this later statement is correct.

Discussions

- From the definition of O, it should be clear that f(n)=O(g(n)) is not the same as O(g(n)) = f(n).
- In fact it is meaningless to say that O(g(n)) = f(n).
- The use of the symbol = is unfortunate because this symbol commonly denotes the equal relation.
- Some of the confusion that results from the use of this symbol (which is standard terminology) can be avoided by reading the symbol = as “is” and not as ”equals”.

Proof

Prove that if $f(n) = a_m n^m + \ldots + a_1 n + a_0$, then $f(n) = O(n^m)$, for $n >= 1$

Proof

Prove that if $f(n) = a_m n^m + \ldots + a_1 n + a_0$, then $f(n) = O(n^m)$, for $n >= 1$

$$f(n) <= \sum_{i=0}^{m} |a_i| \, n^i$$

$$<= n^m \sum_{i=0}^{m} |a_i| \, n^{i-m}$$

$$<= n^m \sum_{i=0}^{m} |a_i| \qquad \text{for } n >= 1$$

Hence, $f(n) = O(n^m)$